Git cheat sheet

The commands work knowing <angle brackets> are values you fill in.

SET UP & FIRST LOOK

`git config --global user.name "Name"`	Set the name on your commits (once per machine)
`git config --global user.email "you@x.com"`	Set the email on your commits
`git init`	Start a repository in the current folder
`git clone <url>`	Copy a remote repository to your machine
`git status`	What changed, what is staged (use -s for the short form)
`git log --oneline --graph --all`	Compact history with branches drawn in

SAVE YOUR WORK

`git add <file> ...`	Stage exactly these files for the next commit
`git add -p`	Stage piece by piece, choosing hunks
`git add -N <file>`	Mark a brand-new file as intended, so a path commit sees it
`git commit -m "message"`	Commit what is staged
`git commit <file> ... -m "msg"`	Commit only the named files, ignoring everything else staged
`git commit --amend`	Rewrite the last commit (only if it is not pushed)
`git restore --staged <file>`	Unstage a file, keeping your edits

LOOK AT CHANGES

`git diff`	Unstaged changes
`git diff --staged`	What the next commit will contain
`git diff main..HEAD`	Everything this branch has that main does not
`git show <commit>`	One commit's message and changes
`git blame <file>`	Who last touched each line
`git log -p <file>`	History of one file, with the changes
`git log --author="name" --since="2 weeks"`	Filter history

BRANCHES

`git branch`	List local branches (-a includes remote ones)
`git switch <branch>`	Move to a branch
`git switch -c <new-branch>`	Create a branch and move to it
`git merge <branch>`	Bring that branch into the current one
`git rebase main`	Replay this branch on top of main (before it is shared)
`git branch -d <branch>`	Delete a merged branch (-D forces it)
`git cherry-pick <commit>`	Copy one commit onto the current branch

REMOTES

`git remote -v`	Show where origin points
`git fetch`	Download new commits without changing your files
`git pull`	Fetch, then merge into the current branch
`git push`	Upload your commits
`git push -u origin <branch>`	First push of a new branch, and remember it
`git push --force-with-lease`	Safer forced push: refuses if someone else pushed

UNDO & RECOVER

`git restore <file>`	Throw away edits to a file (cannot be undone)
`git revert <commit>`	Undo a commit by adding a new one (safe when pushed)
`git reset --soft HEAD~1`	Undo the last commit, keep changes staged
`git reset --hard <commit>`	Move the branch and DISCARD changes (dangerous)
`git stash / git stash pop`	Shelve uncommitted work, bring it back later
`git reflog`	Every place HEAD has been - how you find a 'lost' commit
`git clean -n`	Preview untracked files that would be deleted (-fd to do it)

TAGS, IGNORING & HOUSEKEEPING

`git tag v1.0`	Mark a commit (git push --tags to publish)
`git rm --cached <file>`	Stop tracking a file but keep it on disk
`git ls-files --others --exclude-standard`	List untracked files that are not ignored
`git check-ignore -v <path>`	Which .gitignore rule is ignoring this path
`git gc`	Tidy and compress the repository

Rule of thumb: commands that only look (status, diff, log, show) are always safe. Anything with --hard, --force, -D or clean -f can destroy work - check `git status` *first.*